

Отчёт по практической работе №1 — Пользовательские запросы в веб-приложении (DevTools, HTTP, REST, JSON, Postman)

ЗАДАНИЕ 1.

Name	Status	Type	Initiator	Size	Time
posts	200	document	Other	7.2 kB	193 ms
favicon.ico	200	x-icon	Other	0.2 kB	184 ms

- метод;

GET

- URL и параметры;

<https://jsonplaceholder.typicode.com/posts>

Параметров нет

- заголовки (Request Headers / Response Headers);

▼ Response headers	
Access-Control-Allow-Credentials	true
Age	12552
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
CF-Cache-Status	HIT
CF-Ray	a3858985eeadd9c3-ARN
Content-Encoding	gzip
Content-Type	application/json; charset=utf-8
Date	Wed, 09 Sep 2026 10:33:52 GMT
Etag	W/"6b80-Ybsq/K6GwqYKASFXqDXGC7DoH"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=AH21K1C1J3E1E2981EXUvtgY1IoLCQ%2F5G%2F4nW9juxo%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1785332330"}],"max_age":3600}
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=AH21K1C1J3E1E2981EXUvtgY1IoLCQ%2F5G%2F4nW9juxo%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1785332330"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=7,cfOrigin;dur=0
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000
X-Ratelimit-Remaining	999
X-Ratelimit-Reset	1785332363

▼ Request Headers	
:authority	jsonplaceholder.typicode.com
:method	GET
:path	/posts
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7
Cache-Control	no-cache
Pragma	no-cache
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="152", "Not?A_Brand";v="24", "Google Chrome";v="152"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36

● тело ответа (Response / Preview);

	Headers	Preview	Response	Initiator	Timing
1	[[				
2	{				
3	"userId": 1,				
4	"id": 1,				
5	"title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",				
6	"body": "quia et suscipit\nsuscepit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet ai				
7	},				
8	{				
9	"userId": 1,				
10	"id": 2,				
11	"title": "qui est esse",				
12	"body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciend:				
13	},				
14	{				
15	"userId": 1,				
16	"id": 3,				
17	"title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",				
18	"body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labor				
19	},				
20	{				
21	"userId": 1,				
22	"id": 4,				
23	"title": "eum et est occaecati",				
24	"body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iui				
--	,				

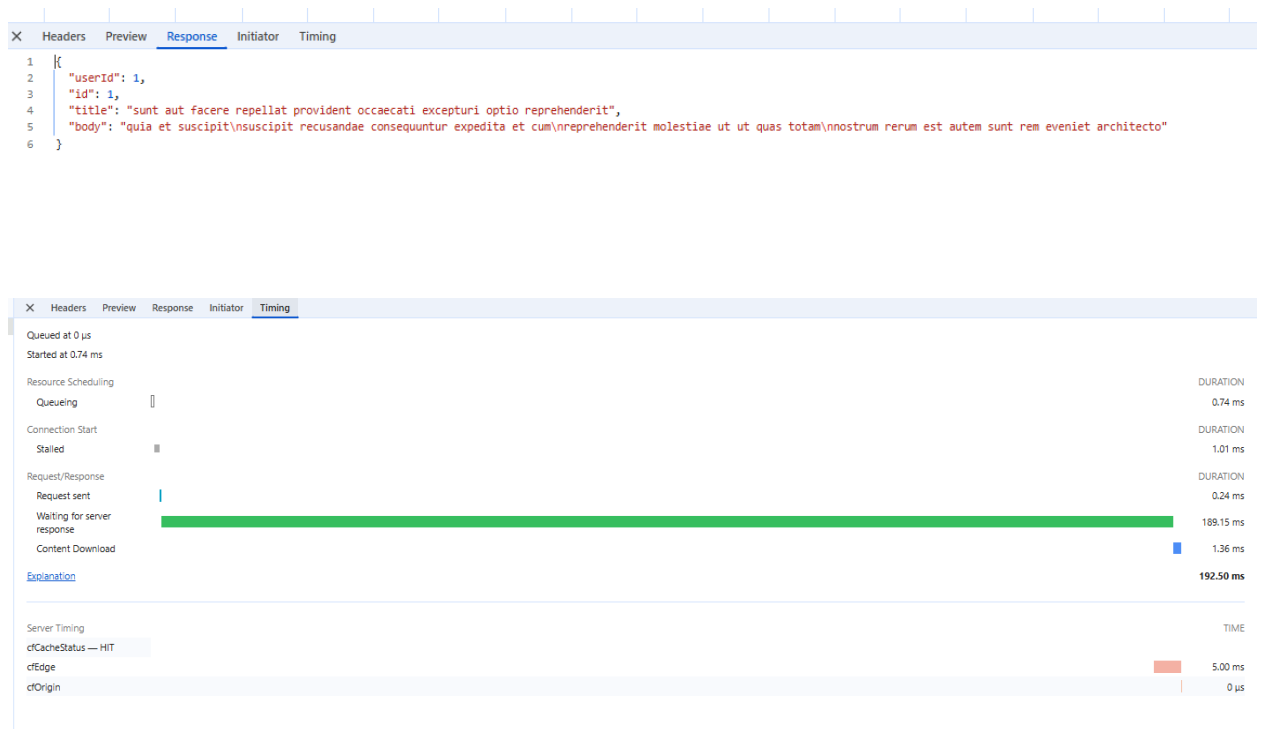
	Headers	Preview	Response	Initiator	Timing
▼	[,...]				
▶ 0:	{userId: 1, id: 1, title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",...}				
▶ 1:	{userId: 1, id: 2, title: "qui est esse",...}				
▶ 2:	{userId: 1, id: 3, title: "ea molestias quasi exercitationem repellat qui ipsa sit aut",...}				
▶ 3:	{userId: 1, id: 4, title: "eum et est occaecati",...}				
▶ 4:	{userId: 1, id: 5, title: "nesciunt quas odio",...}				
▶ 5:	{userId: 1, id: 6, title: "dolorem eum magni eos aperiam quia",...}				
▶ 6:	{userId: 1, id: 7, title: "magnam facilis autem",...}				
▶ 7:	{userId: 1, id: 8, title: "dolorem dolore est ipsam",...}				
▶ 8:	{userId: 1, id: 9, title: "nesciunt iure omnis dolorem tempora et accusantium",...}				
▶ 9:	{userId: 1, id: 10, title: "optio molestias id quia eum",...}				
▶ 10:	{userId: 2, id: 11, title: "et ea vero quia laudantium autem",...}				
▶ 11:	{userId: 2, id: 12, title: "in quibusdam tempore odit est dolorem",...}				
▶ 12:	{userId: 2, id: 13, title: "dolorum ut in voluptas mollitia et saepe quo animi",...}				
▶ 13:	{userId: 2, id: 14, title: "voluptatem eligendi optio",...}				
▶ 14:	{userId: 2, id: 15, title: "eveniet quod temporibus",...}				
▶ 15:	{userId: 2, id: 16, title: "sint suscipit perspiciatis velit dolorum rerum ipsa laboriosam odio",...}				
▶ 16:	{userId: 2, id: 17, title: "fugit voluptas sed molestias voluptatem provident",...}				
▶ 17:	{userId: 2, id: 18, title: "voluptate et itaque vero tempora molestiae",...}				
▶ 18:	{userId: 2, id: 19, title: "adipisci placeat illum aut reiciendis qui",...}				

● код ответа;

200 OK

● время выполнения.

	X	Headers	Preview	Response	Initiator	Timing
Queued at 0 μ s						
Started at 0.72 ms						
Resource Scheduling						
Queueing						DURATION 0.72 m
Connection Start						
Stalled						DURATION 0.64 m
Request/Response						
Request sent						DURATION 0.25 m
Waiting for server response						190.55 m
Content Download						1.33 m
Explanation						193.50 m
Server Timing						
cfCacheStatus — HIT						TIME
cfEdge						7.00 m
cfOrigin						0 μ



ЗАДАНИЕ №2

- GET /posts (все посты);
- GET /posts/1 (один пост).

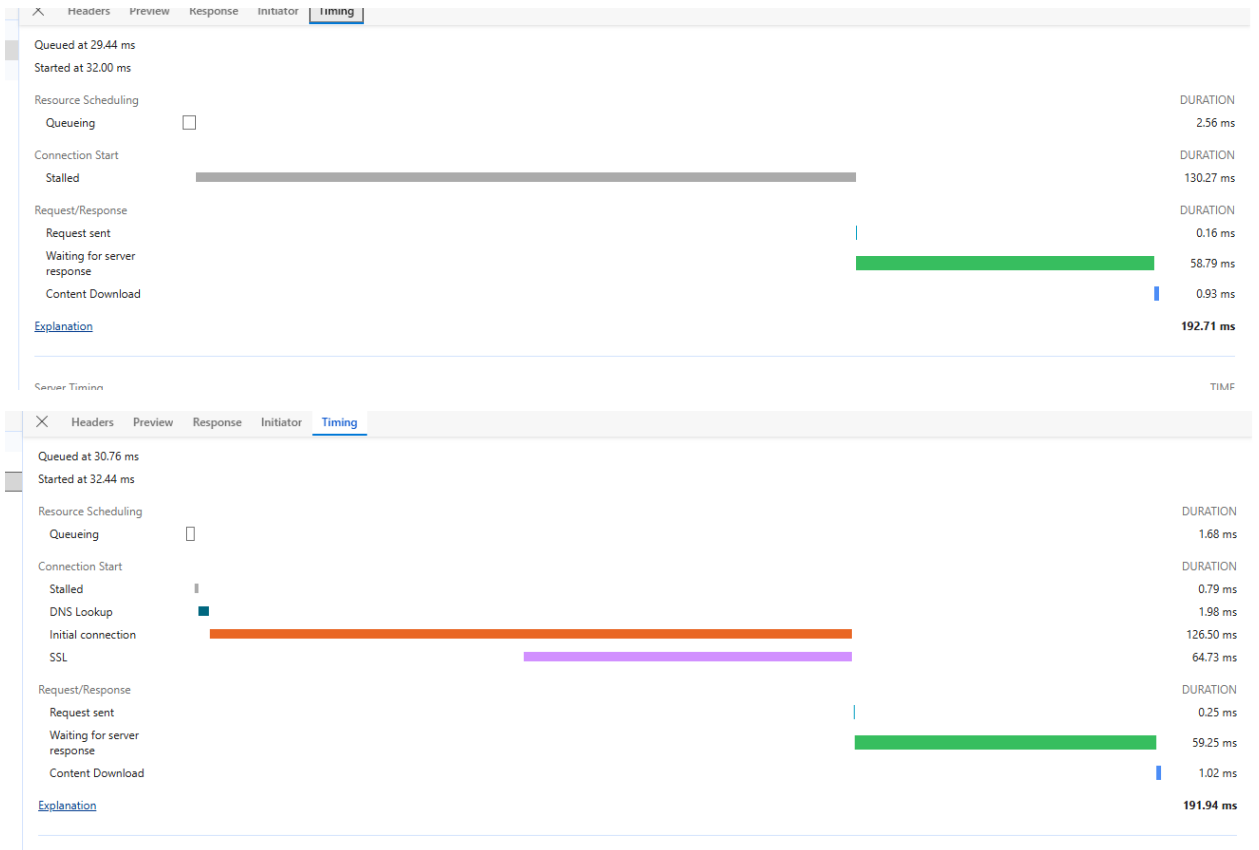
Сделайте выводы по:

- размеру ответа;

index.html	200	document	Other	472 B	3 ms
1	304	fetch	index.html:10	77 B	62 ms
posts	304	fetch	index.html:12	53 B	63 ms

	Size	Time	Fulfil
	472 B	3 ms	
	77 B	62 ms	
	53 B	63 ms	

- времени загрузки;



● количеству записей.

```
▼ {userId: 1, id: 1, title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', body: 'quia et suscipit\nsuscipit recusandae consequuntur \strum rerum est autem sunt rem eveniet architect o'}  
  body: "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"  
  id: 1  
  title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit"  
  userId: 1  
  ▶ [[Prototype]]: Object  
  
▼ (100) [{"id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsuscipit recusandae consequuntur \strum rerum est autem sunt rem eveniet architect o"}, {"id": 2, "title": "qui est esse", "body": "est rerum tempore vitae\nsequi sint nihil reprehend\naperiam non debitis possimus qui neque nisi nulla"}, {"id": 3, "title": "eum et est occaecati", "body": "ullam et saepe reiciendis voluptatem adipisci\nsit \n ipsam iure\nquis sunt voluptatem rerum illo velit"}, {"id": 4, "title": "nesciunt quas odio", "body": "repudiandae veniam quaerat sunt sed\nnullaiaa fugi\nsse voluptatibus quis\nnest aut tenetur dolor neque"}, {"id": 5, "title": "dolorem eum magni eos aperiam quia", "body": "ut aspernatur corporis harum nihil quis provident \s\nvoluptate dolores velit et doloreque molestiae"}]
```

2. сколько объектов вернулось;

1 объект

	headers	Payload	Preview	Response	Initiator	Timing
1				[		
2				{		
3				"userId": 1,		
4				"id": 1,		
5				"title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",		
6				"body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut		
7				},		
8				{		
9				"userId": 1,		
10				"id": 2,		
11				"title": "qui est esse",		
12				"body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat bli		
13				},		
14				{		
15				"userId": 1,		
16				"id": 3,		
17				"title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",		
18				"body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accus		
19				},		
20				{		
21				"userId": 1,		
22				"id": 4,		
23				"title": "eum et est occaecati",		
24				"body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nq		
25				},		
26				{		
27				"userId": 1,		
28				"id": 5,		
29				"title": "nesciunt quas odio",		
30				"body": "repudiandae veniam quaerat sunt sed\nalias aut fugiat sit autem sed est\nvoluptatem omnis possim		
31				},		
32				{		
33				"userId": 1,		
34				"id": 6,		
35				"title": "dolorem eum magni eos aperiam quia",		
36				"body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\npersp		
37				},		
38				{		
39				"userId": 1,		

100 объектов

```

    "userId": 1,
    "id": 9,
    "title": "nesciunt iure omnis dolorem tempora et accusantium",
    "body": "consectetur animi nesciunt iure dolore\nenim quia ad\nveniam autem ut quam aut nobis\net est aut
  },
  {
    "userId": 1,
    "id": 10,
    "title": "optio molestias id quia eum",
    "body": "quo et expedita modi cum officia vel magni\ndoloribus qui repudiandae\nvero nisi sit\nquos venia
  },
  {
    "userId": 2,
    "id": 11,
    "title": "et ea vero quia laudantium autem",
    "body": "delectus reiciendis molestiae occaecati non minima eveniet qui voluptatibus\naccusamus in eum be
  },
  {
    "userId": 2,
    "id": 12,
    "title": "in quibusdam tempore odit est dolorem",
    "body": "itaque id aut magnam\npraesentium quia et ea odit et ea voluptas et\nsapiente quia nihil amet oc
  },
  {
    "userId": 2,
    "id": 13,

```

3. размер ответа;

4. время загрузки.

Name	Status	Type	Initiator	Size	Time
posts?userId=1	200	document	Other	1.4 kB	187 ms

Name	Status	Type	Initiator	Size	Time
posts	200	document	Other	7.8 kB	186 ms

Сделайте вывод, как параметры помогают **уменьшить объём данных**. — чем больше фильтров, тем меньше объём данных

ЗАДАНИЕ №3

- как меняется время отклика;

Name	Status	Type	Initiator	Size	Time	Fulfilled by
index.html	200	document	Other	584 B	4 ms	
1	304	fetch	index.html:10	77 B	72 ms	
3	304	fetch	index.html:12	53 B	72 ms	
2	304	fetch	index.html:11	53 B	73 ms	

- как браузер параллелит запросы;

Браузер выполняет вопросы одновременно.

Второй вопрос выполняет с погрешностью в миллисекунду.

браузер не ждёт, пока завершится первый запрос, чтобы начать второй

- нет ли избыточных запросов.

2 — 53 В

3 — 53 В

Все ответы — **микроскопические** (меньше 100 байт).

2.7. Заголовки и ответы

1. Во вкладке **Headers** выпишите не менее 5 ключевых заголовков, например:

□ **Content-Type** — указывает тип передаваемых данных (например, `text/html` для веб-страниц или `application/json` для данных), чтобы браузер знал, как правильно отобразить информацию.

□ **Cache-Control** — управляет кэшированием. Он говорит браузеру, можно ли сохранять страницу или файл на компьютере и как долго их использовать повторно без обращения к серверу.

□ **ETag** (Entity Tag) — уникальный цифровой код конкретной версии файла. Если файл не менялся, код остается прежним, что помогает загружать данные быстрее.

□ **Date** — показывает точное время и дату, когда сервер отправил этот ответ.

□ **Server** — содержит информацию о программном обеспечении самого веб-сервера (например, *Nginx* или *Apache*).

(HTTP/1.1	200	OK					
			Content-Type:	application/json	←	Формат данных	
			Content-Length:	77	←	Размер ответа	
			Cache-Control:	max-age=43200	←	Кэшировать	
			ETag:	"W/6b80-..."	←	Версия файла	
			Date:	Wed, 09 Sep 2026 11:57:49 GMT	←	Когда отправлен	
			Server:	cloudflare	←	Кто ответил	

(	GET	/posts/1	HTTP/1.1					
			Host:	jsonplaceholder.typicode.com	←	Куда		
			User-Agent:	Mozilla/5.0	←	Кто (браузер)		
			Accept:	application/json	←	Что хочу		

| | | Accept-Encoding: gzip, deflate ← Как сжимать | | |
| | | Connection: keep-alive ← Держать связь)

7. Сравните вкладки **Preview** и **Response**:

Вкладки **Preview** (Просмотр) и **Response** (Ответ) находятся в панели разработчика браузера (вкладка Network) и показывают данные, пришедшие от сервера.

чем они отличаются по представлению данных

Главное отличие — **формат отображения**

в каких случаях удобнее использовать Preview, а в каких — Response.

	Content-Type	Cache-Control	ETag	Date	Server
https://jsonplaceholder.typicode.com/posts	application/json; charset=utf-8	max-age=3200	W/"6b80-Ybsq/K6GwwqrYkAs FxqDXGC7DoM"	Wed, 09 Sep 2026 11:57:49 GMT	cloudflare

ЗАДАНИЕ 4

2.9. Работа с API через Postman

1. Перейдите на сайт:

<https://www.postman.com/>

1. Выполните:

2. GET /posts

Pasted image 20260909151621.png

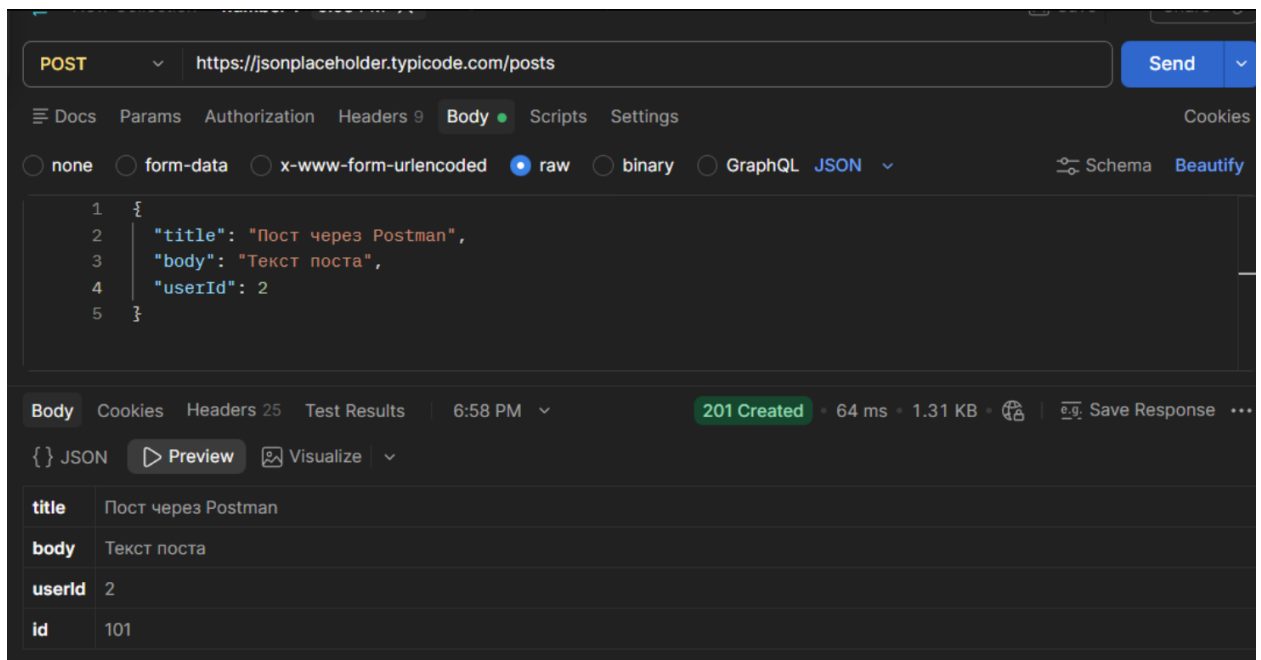
3. GET /posts/1

Pasted image image 20260909151640.png

Сравните ответы.

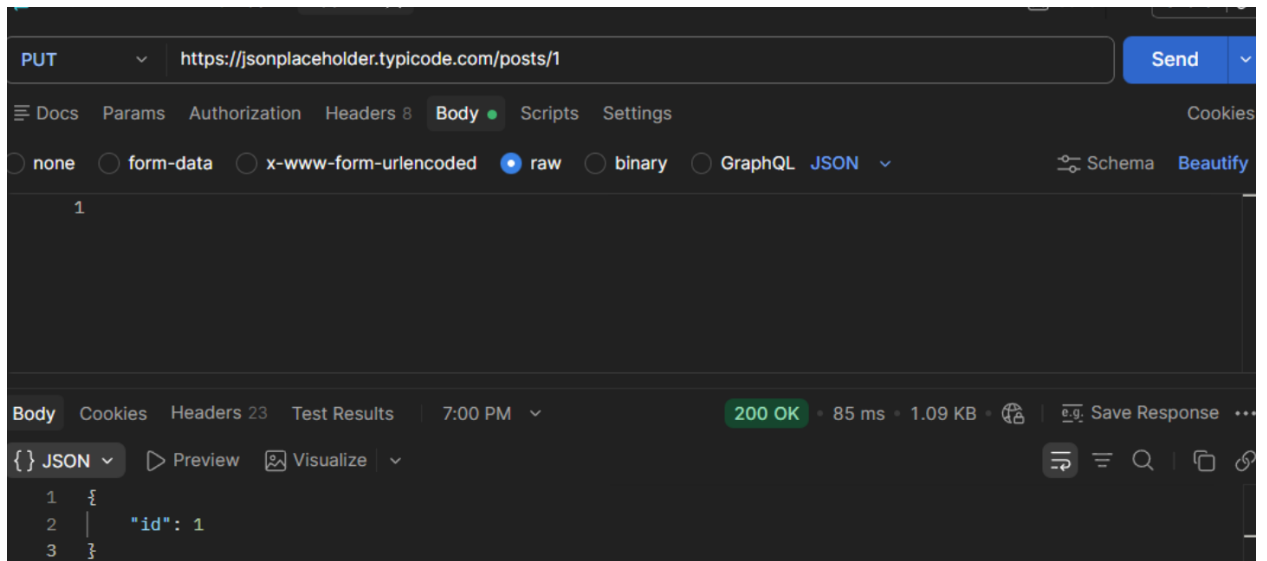
4. Выполните **POST** /posts с JSON-телом:

```
{  
  "title": "Пост через Postman",  
  "body": "Текст поста",  
  "userId": 2  
}
```

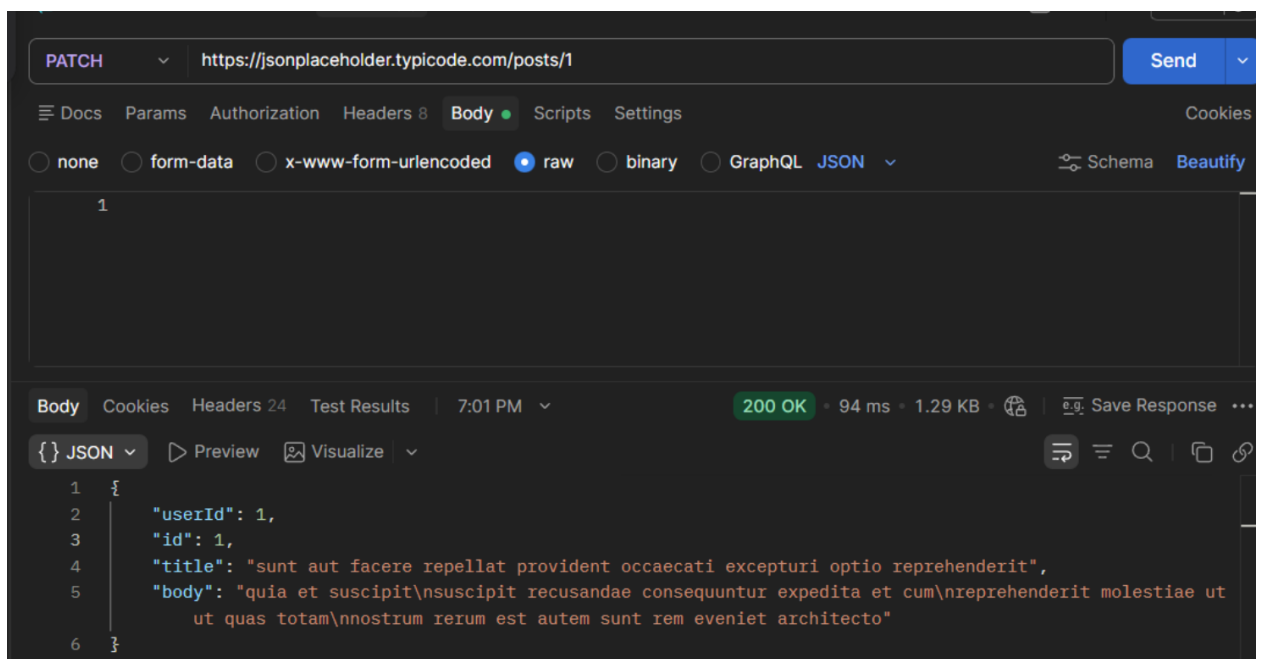


1. Выполните:

2. **PUT** /posts/1 — полное обновление ресурса;

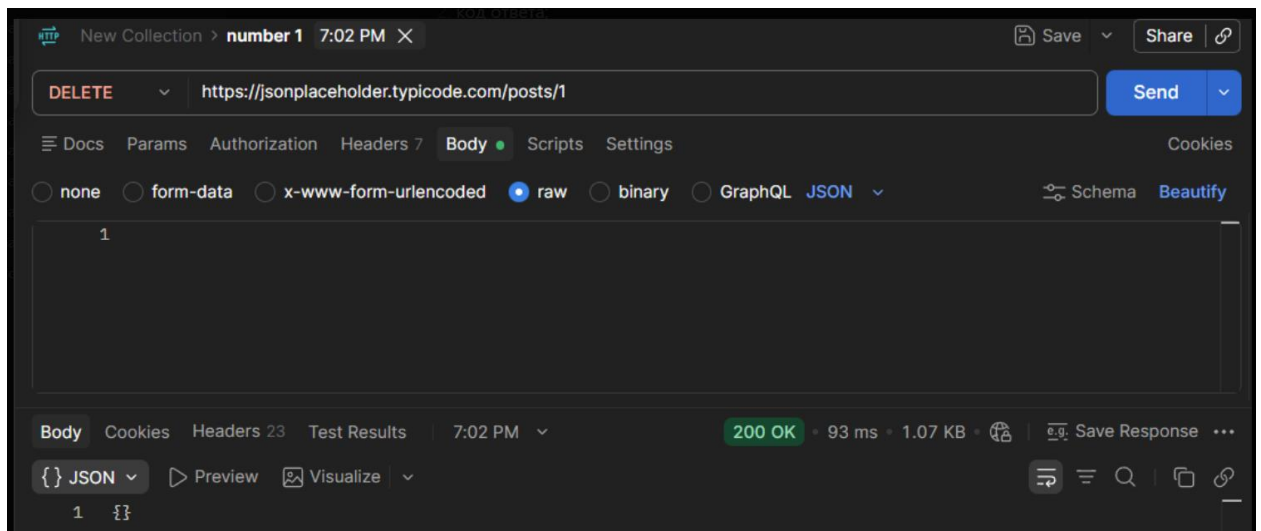


3. **PATCH** /posts/1 — частичное обновление (изменить только один-два поля).



Объясните, чем отличаются результаты.

1. Выполните **DELETE** /posts/1 и зафиксируйте:
2. код ответа;
3. тело ответа (если есть).



1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

DevTools – набор инструментов, вложенный в браузер, позволяющий исследовать веб-страницу, отслеживать сетевой трафик, измерять производительность.

Elements – показывает DOM-дерево страницы и CSS-стили. Позволяет в реальном времени редактировать HTML и CSS, чтобы посмотреть, как изменения отразятся на внешнем виде, а также находить элементы для написания селекторов.

Console – отображает логи JavaScript (console.log), ошибки и предупреждения. Также служит интерактивной средой (REPL), где можно выполнять JS-код прямо на странице.

Network - показывает все сетевые запросы, которые делает страница (загрузка HTML, CSS, картинок, API-запросы). Позволяет смотреть заголовки, тело запроса/ответа, время выполнения и статус-коды.

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

- 1) Клавиша F12.
- 2) Сочетание Ctrl+Shift+I (Windows/Linux).
- 3) Правой кнопкой мыши по странице -> «Исследовать элемент» (Inspect).

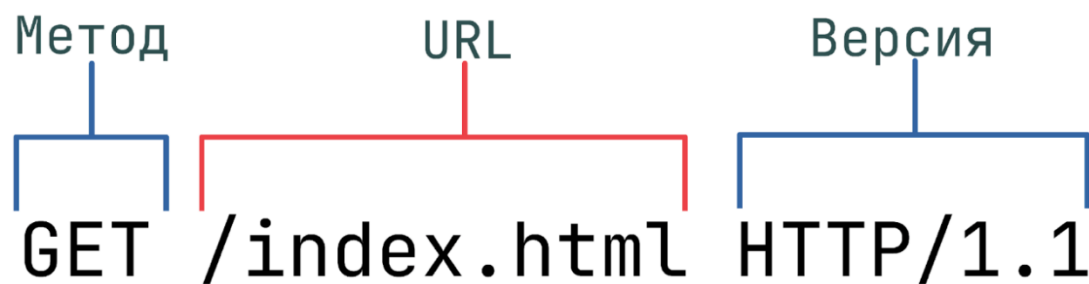
Disable cache (Отключить кэш): Обязательно ставить галочку, чтобы видеть реальное время загрузки всех ресурсов, а не брать их из локального кэша.

Throttling (Симуляция скорости): Позволяет эмулировать медленное соединение (3G, Slow 4G), чтобы понять, как сайт поведет себя у пользователей с плохим интернетом.

Фильтры (Fetch/XHR): Позволяют скрыть лишние ресурсы (картинки, шрифты) и оставить только API-запросы.

Waterfall (Каскад): Визуализация времени загрузки каждого запроса. Важно смотреть на "Stalled", "Waiting (TTFB)" и "Content Download".

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.



Метод (HTTP Method): Указывает серверу, какое действие нужно совершить над ресурсом (например, получить, создать, обновить).

URL и параметры: Адрес целевого ресурса. Параметры строки запроса (после знака ?, например, ?search=phone&page=2) используются для фильтрации или сортировки данных.

Заголовки (Headers): Служебная метаинформация в формате «ключ-значение». Передают данные о типе контента, кодировке, авторизации, браузере пользователя (User-Agent) и настройках кэша.

Тело (Body / Payload): Сами данные, которые отправляются на сервер. Используется в методах, создающих или изменяющих ресурсы. Чаще всего передается в формате JSON, XML или Form Data.

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

GET — получение данных. Безопасный, идемпотентный (повторение не меняет результат). Пример: Загрузка списка товаров GET /api/products.

POST — создание нового ресурса. Не идемпотентен (два одинаковых запроса создадут две записи). Пример: Отправка формы регистрации POST /api/users.

PUT — полное обновление ресурса. Идемпотентен. Пример: Замена всех данных профиля пользователя PUT /api/users/1.

PATCH — частичное обновление ресурса. Пример: Смена только email пользователя PATCH /api/users/1.

DELETE — удаление ресурса. Идемпотентен. Пример: Удаление комментария DELETE /api/comments/5.

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?

Методы HEAD и OPTIONS используются для получения метаданных без загрузки основного содержимого, а preflight-запрос встречается в механизме безопасности веб-браузеров.

Метод HEAD

Назначение — запрашивает только заголовки ответа, полностью идентичные тем, что вернул бы метод GET.

Тело ответа — всегда пустое.

Применение — проверка существования ресурса, сканирование ссылок поисковыми роботами.

Применение — проверка размера файла через заголовок Content-Length перед его скачиванием.

Применение — проверка даты изменения ресурса через Last-Modified для актуализации кэша.

Метод OPTIONS

Назначение — запрашивает параметры соединения или доступные свойства конкретного ресурса.

Тело ответа — обычно пустое, но сервер возвращает список поддерживаемых методов в заголовке Allow (например, Allow: GET, POST, OPTIONS).

Применение — определение возможностей сервера без выполнения реального действия над данными.

Где встречается preflight-запрос?

Контекст — preflight-запрос (предварительный запрос) встречается исключительно в механизме CORS (Cross-Origin Resource Sharing) в веб-браузерах.

Триггер — браузер отправляет его автоматически перед выполнением «сложного» междоменного запроса (например, с использованием методов PUT, DELETE или кастомных заголовков вроде Authorization).

Суть — браузер делает автоматический запрос методом OPTIONS, чтобы узнать у сервера, разрешено ли сайту с другого домена выполнять основную операцию.

Результат — если сервер отвечает согласием (заголовки Access-Control-Allow-Origin и Access-Control-Allow-Methods), браузер отправляет реальный запрос (POST, PUT и т.д.).

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

1xx (Информационные): Запрос получен, продолжается обработка.

2xx (Успех): Действие успешно принято - 200 OK: Стандартный успешный ответ.

3xx (Перенаправление): Для выполнения запроса нужны дальнейшие действия - 301 Moved Permanently: Ресурс переехал навсегда (нужно обновить ссылку), 302 Found: Временное перенаправление, 304 Not Modified: Ресурс не изменился, можно использовать кэш браузера.

4xx (Ошибка клиента): Проблема в запросе (неверный синтаксис, нет прав) - 404 Not Found: Ресурс не найден.

5xx (Ошибка сервера): Сервер не смог выполнить корректный запрос - 500 Internal Server Error: Ошибка на стороне сервера (баг в коде).

7. Что такое REST и как ресурс в REST представлен на уровне URL?

REST API (Representational State Transfer Application Programming Interface) — это архитектурный стиль взаимодействия между клиентом и сервером через HTTP. Он определяет принципы построения API, обеспечивая стандартизированный и эффективный обмен данными между различными системами.

Представлен как посредник между клиентом и сервером

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

JSON поддерживает 6 основных типов данных: строку, число, логическое значение, null, массив и объект.

```
{ "name": "Алексей", "age": 25, "isStudent": false }
```

```
[  
  "яблоко",  
  "банан",  
  "пельмени"  
]
```

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Главное отличие между этими вкладками — формат отображения данных. Вкладка Response показывает сырой ответ сервера, а Preview форматирует его для удобного чтения.

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

Управление кэшированием (Снижение нагрузки на сервер) Cache-Control: Задаёт правила кэширования (например, max-age, public, no-store). ETag: Токен версии ресурса. Позволяет серверу не пересылать файл, если он не менялся. Last-Modified: Дата изменения файла. Используется для проверки

актуальности контента. Expires: Устаревший, но все еще используемый заголовок даты окончания актуальности кеша. Сжатие и тип данных (Уменьшение размера ответов)Content-Encoding: Указывает алгоритм сжатия данных (например, br для Brotli или gzip). Content-Type: Определяет MIME-тип ресурса (например, image/webp). Помогает проверить, оптимизированы ли форматы. Content-Length: Размер тела ответа в байтах. Нужен для контроля избыточного веса страниц. Безопасность и авторизация (Оптимизация оверхеда) Authorization: Передача токенов (например, Bearer). Проверяется, чтобы заголовки не ломали публичное кеширование. Vary: Указывает, какие заголовки запроса определяют уникальность копии в кеше (например, Vary: Accept-Encoding). Неправильная настройка ломает кеш. Сетевое взаимодействие Connection: Управление постоянным соединением (значение keep-alive снижает время на повторное TLS-рукопожатие).

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

Пагинация — разделение больших списков на части (страницы) с помощью курсоров (cursor) или смещений (offset).

Фильтрация — передача только тех записей, которые соответствуют критериям пользователя (например, ?status=active).

Кэширование — сохранение ответов на стороне клиента, прокси или CDN с помощью заголовков Cache-Control и ETag.

Сжатие данных — уменьшение размера тела ответа алгоритмами Gzip или Brotli через заголовок Content-Encoding.Проекция полей — запрос только необходимых свойств объекта (выборка полей, например, ?fields=id,name).

Ограничение частоты (Rate Limiting) — защита от перегрузок путем лимитирования количества запросов от одного клиента.

Пакетные запросы (Batching) — объединение нескольких мелких запросов в один сетевой пакет.

Веб-сокеты / Веб-хуки — замена постоянных запросов (polling) на событийно-ориентированное обновление данных.

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

Критерий	DevTools (Вкладка Network)	Postman
Основная цель	Анализ текущих запросов страницы	Создание, отладка и автоматизация API
Редактирование	Сложно изменить и отправить запросов заново	Легко менять любые параметры, заголовки и тело
Автоматизация	Нет	Написание тестов, запуск коллекций (Runner)
Сохранение	Пропадает после закрытия вкладки	Сохранение в коллекции, синхронизация в облаке
Окружение	Зависит от браузера и куки сессии	Настройка переменных окружения (Env: Dev, Prod)